同芯同行

Chip in, big win

UVM Scoreboard 常考题 / 真题汇总

Part 2 / 3 · 题目 10 ~ 17

SystemVerilog · UVM 1.2 / IEEE 1800.2

本册涵盖题目 10-17

流水线drain-time（10）/ 以太网错误注入（11）/ 参数化泛型（12）

CDC跨域同步（13）/ SRAM内存模型（14）/ 可扩展回调（15）

图像DPI-C对接（16）/ UDP协议拆包（17）

同芯同行，真题先行

题目 10 带 phase_ready_to_end 的流水线 Scoreboard

难度：高级

标签：phase_ready_to_end / drain-time / raise_objection / 流水线排空

【题目描述】

验证一个 4 级流水线处理器。当激励序列停止发送之后，DUT 内部仍有若干 in-flight 事务在各流水线级之间传递，这些事务需要等待全部流出并完成比对后，验证才算正式结束。如果不做特殊处理，UVM 的 run_phase 会在激励停止时立即退出，导致流水线中的残留事务未被验证就被丢弃，产生验证漏洞。本题考察如何使用 phase_ready_to_end 机制延迟 run_phase 的结束，等待流水线完全排空，同时通过超时保护避免因 DUT 故障导致仿真永久挂起。

【考察重点与实现要点】

> phase_ready_to_end 在 run_phase 即将结束时（每次激励停止时）被 UVM 调用
> raise_objection 阻止 phase 结束；drop_objection 允许 phase 结束，两者必须成对
> join_none 启动后台 drain task，不阻塞 phase_ready_to_end 本身的返回
> 超时保护是防止死锁的关键：若 DUT 故障导致事务永远不能流出，超时后强制 drop
> write() 中 is_input 时 pending_cnt++，输出侧事务时 pending_cnt--（配合功能比对）

【完整答案代码】

```
// 方法论：input（流水线入口）→ analysis_fifo 缓存期望值
// output（流水线出口）→ imp，write() 中从 fifo 取期望值立即比对
// 流水线延迟导致仿真结束时 fifo 可能非空，用 phase_ready_to_end 排空
class pipeline_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(pipeline_scoreboard)

  // 输入侧：analysis_fifo 缓存入口事务（含期望结果）
  uvm_analysis_export #(pipe_item) input_export;
  uvm_tlm_analysis_fifo #(pipe_item) input_fifo;

  // 输出侧：imp，write() 立即从 fifo 取对应期望比对
  uvm_analysis_imp #(pipe_item, pipeline_scoreboard) output_imp;

  int unsigned DRAIN_TIMEOUT = 1000; // ns

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    input_export = new("input_export", this);
    input_fifo = new("input_fifo", this);
    output_imp = new("output_imp", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    input_export.connect(input_fifo.analysis_export);
  endfunction

  // output 端：write() 从 fifo 取对应入口事务，立即比对结果
  function void write(pipe_item act);
    pipe_item inp;
    if (!input_fifo.try_get(inp)) begin
      `uvm_error("SB", "流水线出口收到数据，但入口 fifo 为空！")
      return;
    end
    if (act.result !== inp.expected)
      `uvm_error("SB", $sformatf(
```

```
        "流水线结果错误: exp=0x%0h act=0x%0h",
        inp.expected, act.result))
  endfunction

  // 考点：激励停止时 fifo 可能仍有残留事务（流水线延迟），
  // 用 phase_ready_to_end 等待 fifo 排空
  function void phase_ready_to_end(uvm_phase phase);
    if (phase.get_name() != "run") return;
    if (input_fifo.size() > 0) begin
      phase.raise_objection(this,
        $sformatf("等待 %0d 个流水线事务排空", input_fifo.size()));
      fork
        wait_for_drain(phase);
      join_none
    end
  endfunction

  task wait_for_drain(uvm_phase phase);
    fork
      begin : wait_done
        wait(input_fifo.size() == 0);
        `uvm_info("SB", "流水线已全部排空", UVM_LOW)
      end
      begin : timeout_guard
        #(DRAIN_TIMEOUT * 1ns);
        if (input_fifo.size() > 0)
          `uvm_error("SB", $sformatf(
            "Drain 超时！fifo 仍有 %0d 个事务未流出", input_fifo.size()))
      end
    join_any
    disable fork;
    phase.drop_objection(this, "流水线排空结束");
  endtask
endclass
```

【关键提示】

input 侧用 analysis_fifo，output 侧用 imp，write() 立即 try_get 并比对。流水线延迟使 fifo 在激励停止后仍有残留，phase_ready_to_end 等待 fifo.size() == 0 后才允许 run_phase 结束，比原来用 pending_cnt 更自然。

题目 11 协议错误注入验证 Scoreboard

难度：初级

标签：负向测试 / 错误注入 / 双向验证 / dut_err_flag

【题目描述】

验证以太网控制器的错误帧检测能力。验证场景分为两类：一类发送符合 IEEE 802.3 规范的正常帧，期望 DUT 正常放行；另一类主动注入各种类型的错误帧（包括 CRC 错误、帧长越界、前导码错误等），期望 DUT 检测到错误并丢弃该帧，同时置位对应的错误标志。Scoreboard 必须同时保证两个方向的正确性：正常帧不能被 DUT 误判为错误（false positive），错误帧不能被 DUT 漏检放行（false negative）。本题考察负向测试中双向验证设计的必要性与实现方式。

【考察重点与实现要点】

> has_error 是激励意图（sequence 设置），dut_err_flag 是 DUT 响应（硬件输出），不能混淆
> 双向验证：正常帧不误报（false positive）和错误帧不漏报（false negative）缺一不可
> err_cnt[string] 按错误类型统计，report_phase 中打印，验证各类错误都被覆盖到
> check_payload() 验证帧长（46~1500字节）和 MAC 地址合法性，是 IEEE 802.3 协议要求
> write() 中仅分流，不做任何比对，让 run_phase 做所有阻塞式比对

【完整答案代码】

```
class eth_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(eth_scoreboard)

  uvm_analysis_imp #(eth_item, eth_scoreboard) analysis_export;

  eth_item rx_q[$]; // 正常帧队列
  eth_item err_q[$]; // 错误注入帧队列
  int err_cnt[string]; // 按错误类型统计检测次数

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
  endfunction

  // 考点：按激励意图分流，不在 write() 中比对
  function void write(eth_item pkt);
    if (pkt.has_error) err_q.push_back(pkt);
    else rx_q.push_back(pkt);
  endfunction

  task run_phase(uvm_phase phase);
    eth_item pkt;
    phase.raise_objection(this);
    forever begin
      wait(rx_q.size() > 0 || err_q.size() > 0);

      // 处理正常帧：DUT 不能误报错误
      if (rx_q.size() > 0) begin
        pkt = rx_q.pop_front();
        if (pkt.dut_err_flag)
          `uvm_error("SB", $sformatf(
            "正常帧被 DUT 误判为错误帧！dst=0x%0h", pkt.dst_mac))
        check_payload(pkt);
      end
```

```
      // 处理错误注入帧：DUT 必须检测到错误
      if (err_q.size() > 0) begin
        pkt = err_q.pop_front();
        if (!pkt.dut_err_flag)
          `uvm_error("SB", $sformatf(
            "错误帧未被 DUT 检测到！类型=%s", pkt.err_type))
        else begin
          err_cnt[pkt.err_type]++;
          `uvm_info("SB", $sformatf(
            "正确检测到 %s 错误", pkt.err_type), UVM_LOW)
        end
      end
    end
    phase.drop_objection(this);
  endtask

  function void check_payload(eth_item pkt);
    if (pkt.payload.size() < 46 || pkt.payload.size() > 1500)
      `uvm_error("SB", $sformatf(
        "帧长越界: %0d 字节（合法范围 46~1500）", pkt.payload.size()))
    if (pkt.dst_mac === 48'h0)
      `uvm_error("SB", "目标 MAC 地址为全零，非法帧")
  endfunction

  function void report_phase(uvm_phase phase);
    string t;
    `uvm_info("SB", "
=== 错误注入统计 ===", UVM_NONE)
    if (err_cnt.first(t)) do begin
      `uvm_info("SB", $sformatf(" %s : %0d 次正确检测", t, err_cnt[t]), UVM_NONE)
    end while (err_cnt.next(t));
  endfunction
endclass
```

【关键提示】

TX monitor（发送侧）→ analysis_fifo；RX monitor（接收侧）→ imp。write() 中 try_get 取发送帧，根据 has_error 决定期望行为：正常帧验证 dut_err_flag==0，错误帧验证 dut_err_flag==1，逻辑清晰，无需两个独立队列。

题目 12 参数化泛型 Scoreboard（模板类）

难度：中级

标签：uvm_component_param_utils / pure virtual / 泛型 / 模板类

【题目描述】

在多个项目中，Scoreboard 的队列管理、期望值匹配、统计计数等框架代码高度重复，只有协议特定的参考模型计算和数据比对逻辑各不相同。需要设计一个参数化的泛型 Scoreboard 基类，通过类型参数（REQ_T 和 RSP_T）来适配不同协议的请求和响应事务类型，子类只需实现两个协议特定的虚函数：参考模型计算函数 predict() 和数据比对函数 compare()，即可完成一个完整的协议 Scoreboard。本题考察 UVM 参数化组件的声明方式，以及纯虚函数（pure virtual）在框架复用设计中的作用。

【考察重点与实现要点】

> 参数化组件必须用 uvm_component_param_utils，否则工厂注册失败
>
> pure virtual 强制子类实现，编译器保证不遗漏，比文档约定更可靠
>
> virtual class（virtual 修饰 class 关键字）不能被直接实例化，只有具体子类可以
>
> 子类通过 typedef sb_base #(uart_req, uart_rsp) super_t 简化父类引用
>
> add_expected() 由 reference model 或 sequence 调用，write() 由分析端口调用

【完整答案代码】

```
// ===== 第一层：参数化泛型基类 =====
class sb_base #(type REQ_T = uvm_sequence_item,
                type RSP_T = uvm_sequence_item)
               extends uvm_scoreboard;

  typedef sb_base #(REQ_T, RSP_T) this_type;
  // 考点：参数化组件专用宏，不能用 uvm_component_utils
  `uvm_component_param_utils(this_type)

  uvm_analysis_imp #(RSP_T, this_type) analysis_export;
  REQ_T exp_q[$];
  int match_cnt, mismatch_cnt;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
  endfunction

  // 考点：pure virtual 强制子类实现
  pure virtual function RSP_T predict(REQ_T req);
  pure virtual function bit compare(RSP_T exp, RSP_T act);

  // 框架：外部调用注册期望值
  function void add_expected(REQ_T req);
    exp_q.push_back(predict(req));
  endfunction

  // 框架：write() 调用子类实现的 compare
  function void write(RSP_T actual);
    RSP_T expected;
    if (exp_q.size() == 0) begin
      `uvm_error("SB", "收到响应但期望队列为空") return;
    end
    expected = exp_q.pop_front();
    if (compare(expected, actual)) match_cnt++;
```

```
    else begin
      mismatch_cnt++;
      `uvm_error("SB", "比对失败")
    end
  endfunction

  function void report_phase(uvm_phase phase);
    `uvm_info("SB", $sformatf(
      "通过=%0d 失败=%0d", match_cnt, mismatch_cnt), UVM_NONE)
  endfunction
endclass

// ===== 第二层：UART 专用 Scoreboard =====
class uart_scoreboard extends sb_base #(uart_req, uart_rsp);
  `uvm_component_utils(uart_scoreboard)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  // 实现参考模型：计算期望奇偶校验
  function uart_rsp predict(uart_req req);
    uart_rsp exp = uart_rsp::type_id::create("exp");
    exp.data = req.data;
    exp.parity = ^req.data; // 偶校验
    return exp;
  endfunction

  // 实现比对逻辑
  function bit compare(uart_rsp exp, uart_rsp act);
    if (exp.data !== act.data || exp.parity !== act.parity) begin
      `uvm_error("SB", $sformatf(
        "UART: data exp=0x%0h act=0x%0h parity exp=%0b act=%0b",
        exp.data, act.data, exp.parity, act.parity))
      return 0;
```

```
    end
    return 1;
  endfunction
endclass
```

【关键提示】

参数化组件必须用 uvm_component_param_utils 注册，否则工厂无法识别该类型。pure virtual 函数在基类中不提供实现体，编译器会强制子类实现，比注释约定或文档说明更可靠。virtual class 不能被直接实例化，确保框架不被误用。

题目 13 带 uvm_event 同步的跨域 Scoreboard

难度：高级

标签：CDC / uvm_event_pool / get_global / wait_trigger_data / semaphore

【题目描述】

验证一个跨时钟域（CDC）数据搬运模块。模块将快时钟域的输入数据通过同步器传输至慢时钟域输出。快、慢两个时钟域各有一个 Monitor，分别观测输入和输出数据并发送至同一个 Scoreboard。由于两个时钟域频率不同，慢域数据会在快域数据发出若干个慢时钟周期后才到达，Scoreboard 必须正确处理这一时序差异，在慢域数据稳定后再进行数据比对。本题考察 uvm_event 的跨组件共享机制，以及 semaphore 对 function/task 并发访问共享队列的保护。

【考察重点与实现要点】

> uvm_event.trigger(obj) 可携带 uvm_object 数据，wait_trigger_data() 取出并 $cast
>
> uvm_event_pool::get_global() 确保不同组件共享同一事件对象，而非各自创建独立实例
>
> semaphore 保护 function 和 task 并发访问同一队列，防止数据竞争
>
> 慢域传播延迟用 #(N * CLK_PERIOD * 1ns) 模拟，等待后再取慢域数据比对
>
> write_slow() 只记录最新事务，无需队列，因为 run_phase 会等待后再读

【完整答案代码】

```
`uvm_analysis_imp_decl(_fast)
`uvm_analysis_imp_decl(_slow)

class cdc_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(cdc_scoreboard)

  uvm_analysis_imp_fast #(data_item, cdc_scoreboard) fast_imp;
  uvm_analysis_imp_slow #(data_item, cdc_scoreboard) slow_imp;

  uvm_event data_ready; // 跨组件共享事件
  data_item fast_pkt_q[$];
  data_item slow_pkt;
  semaphore fast_mutex;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    fast_imp = new("fast_imp", this);
    slow_imp = new("slow_imp", this);
    // 考点：get_global 获取全局共享事件，不能 new() 各自创建
    data_ready = uvm_event_pool::get_global("cdc_data_ready");
    fast_mutex = new(1);
  endfunction

  function void write_fast(data_item pkt);
    fast_mutex.get(1);
    fast_pkt_q.push_back(pkt);
    fast_mutex.put(1);
    data_ready.trigger(pkt); // 携带数据触发事件
  endfunction

  function void write_slow(data_item pkt);
    slow_pkt = pkt; // 记录慢域最新事务
  endfunction

  task run_phase(uvm_phase phase);
```

```
    data_item fast_pkt;
    uvm_object obj;
    phase.raise_objection(this);
    forever begin
      // 考点：等待事件并取出携带数据
      data_ready.wait_trigger_data(obj);
      if (!$cast(fast_pkt, obj))
        `uvm_fatal("SB", "事件携带数据类型错误")

      // 等待慢域传播延迟（典型 2~3 个慢时钟周期）
      #(2 * SLOW_CLK_PERIOD * 1ns);

      fast_mutex.get(1);
      if (fast_pkt_q.size() > 0)
        fast_pkt = fast_pkt_q.pop_front();
      fast_mutex.put(1);

      if (slow_pkt == null)
        `uvm_error("SB", "慢域尚未收到数据")
      else if (slow_pkt.data !== fast_pkt.data)
        `uvm_error("SB", $sformatf(
          "CDC 数据错误: fast=0x%0h slow=0x%0h",
          fast_pkt.data, slow_pkt.data))
    end
    phase.drop_objection(this);
  endtask
endclass
```

【关键提示】

uvm_event_pool::get_global() 确保不同组件使用同一事件对象，这是跨组件同步的基础。trigger() 携带数据，wait_trigger_data() 取出后 $cast，是比简单事件更强大的机制。semaphore 保护 write_fast()（function，无法 await）和 run_phase（task，真正并发）之间的队列访问。

题目 14 内存模型 Scoreboard（带地址范围检测）

难度：中级

标签：内存模型 / 字节粒度 / 地址对齐 / 写保护区域

【题目描述】

验证一个 64KB SRAM 控制器，支持字节（BYTE）、半字（HALF）、字（WORD）三种访问宽度，内存内容初始为全 0。Scoreboard 内部需要维护一份完整的软件内存模型：写操作时更新模型，读操作时预测期望返回值并与 DUT 实际输出比对。同时对每笔访问进行三类合法性检查：地址对齐验证（WORD 访问需 4 字节对齐，HALF 需 2 字节对齐）、写保护区域访问检测，以及地址越界检测。本题考察字节粒度内存模型的构建方式，以及不同访问宽度下的小端序数据拼接与分拆。

【考察重点与实现要点】

> 字节粒度存储：WORD 写操作需循环写4个地址，注意小端序（低地址存低字节）
> 对齐检查必须在访问前完成，写保护检查在对齐检查之后、实际写入之前
> read_model() 按访问宽度拼接：WORD = {mem[a+3], mem[a+2], mem[a+1], mem[a]}（小端）
> write() 内多种错误检查互相独立，任一错误不影响其他检查的执行
> 保护区域发现写入时 return，防止污染软件内存模型

【完整答案代码】

```
class sram_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(sram_scoreboard)

  uvm_analysis_imp #(sram_item, sram_scoreboard) analysis_export;

  logic [7:0] mem_model [bit[15:0]]; // 按字节粒度存储

  typedef struct packed {
    bit [15:0] lo; bit [15:0] hi;
  } region_t;
  region_t protected_regions[$] = '{
    '{16'h0000, 16'h00FF}, // 引导 ROM，只读
    '{16'hF000, 16'hFFFF} // 配置区，只读
  };

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
    for (int i = 0; i < 65536; i++) mem_model[i[15:0]] = 8'h00;
  endfunction

  function void write(sram_item pkt);
    // 考点：对齐检查最先做
    if (pkt.size == WORD && pkt.addr[1:0] != 2'b00)
      `uvm_error("SB", $sformatf("WORD 访问未对齐: addr=0x%0h", pkt.addr))
    if (pkt.size == HALF && pkt.addr[0] != 1'b0)
      `uvm_error("SB", $sformatf("HALF 访问未对齐: addr=0x%0h", pkt.addr))

    if (pkt.wr_en) begin
      // 考点：写保护区域检查
      foreach (protected_regions[i])
        if (pkt.addr >= protected_regions[i].lo &&
            pkt.addr <= protected_regions[i].hi) begin
          `uvm_error("SB", $sformatf("写入保护区域！addr=0x%0h", pkt.addr))
          return; // 不更新软件模型
```

```
        end
      // 考点：小端序字节分拆写入
      case (pkt.size)
        BYTE: mem_model[pkt.addr] = pkt.wdata[7:0];
        HALF: begin
          mem_model[pkt.addr] = pkt.wdata[7:0];
          mem_model[pkt.addr+1] = pkt.wdata[15:8];
        end
        WORD: begin
          for (int i = 0; i < 4; i++)
            mem_model[pkt.addr+i[15:0]] = pkt.wdata[8*i+7 -: 8];
        end
      endcase
    end else begin
      logic [31:0] exp = read_model(pkt.addr, pkt.size);
      if (pkt.rdata !== exp)
        `uvm_error("SB", $sformatf(
          "读取不匹配 addr=0x%0h size=%0s exp=0x%0h act=0x%0h",
          pkt.addr, pkt.size.name(), exp, pkt.rdata))
    end
  endfunction

  // 考点：按访问宽度拼接（小端序）
  function logic [31:0] read_model(bit[15:0] addr, access_e sz);
    case (sz)
      BYTE: return {24'h0, mem_model[addr]};
      HALF: return {16'h0, mem_model[addr+1], mem_model[addr]};
      WORD: return {mem_model[addr+3], mem_model[addr+2],
                    mem_model[addr+1], mem_model[addr]};
      default: return 32'hx;
    endcase
  endfunction
endclass
```

【关键提示】

> 字节粒度存储是内存模型的基础，WORD/HALF 访问需分拆/拼接字节，严格遵守小端序（低地址存低字节）。对齐检查、写保护检查、数据比对是三个独立的验证维度，任一发现错误都不影响其他检查的继续执行。

题目 15 基于回调的可扩展 Scoreboard

难度：高级

标签：uvm_callback / uvm_register_cb / uvm_do_callbacks / 开闭原则

【题目描述】

一个已上线的 Scoreboard 需要在不修改源码的情况下，为特定测试用例灵活地插入额外检查逻辑（例如事务延迟超限告警、特定协议规则的二次验证）。直接修改 Scoreboard 源码会影响所有使用该 Scoreboard 的测试用例，存在引入回归缺陷的风险。需要基于 UVM 的回调（callback）机制设计一套可扩展架构，用户通过继承回调基类、实现回调函数，并在 test 中注册回调实例的方式，在完全不修改 Scoreboard 代码的前提下插入自定义检查逻辑。本题考察 uvm_callback 的完整使用链路。

【考察重点与实现要点】

> `uvm_register_cb 必须在 scoreboard class 体内声明，告知 UVM 该类支持哪种回调
>
> `uvm_do_callbacks 的三个参数：组件类型、回调类型、调用的函数（含实参）
>
> pre_compare 可用于前置检查或记录日志；post_compare 可访问比对结果 is_match
>
> 用户回调在 test 中注册：uvm_callbacks#(extensible_sb, sb_callback)::add(sb_inst, cb_inst)
>
> 回调基类的虚函数体为空，未 override 时调用无副作用，保证向后兼容

【完整答案代码】

```
// ===== 回调基类：定义扩展接口 =====
class sb_callback extends uvm_callback;
  `uvm_object_utils(sb_callback)
  function new(string name = "sb_callback"); super.new(name); endfunction

  virtual function void pre_compare(
    uvm_sequence_item exp, uvm_sequence_item act);
  endfunction

  virtual function void post_compare(
    uvm_sequence_item exp, uvm_sequence_item act, bit is_match);
  endfunction
endclass

// ===== 支持回调的 Scoreboard =====
class extensible_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(extensible_scoreboard)
  // 考点：class 体内注册支持的回调类型
  `uvm_register_cb(extensible_scoreboard, sb_callback)

  uvm_analysis_imp #(my_item, extensible_scoreboard) analysis_export;
  my_item exp_q[$];

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
  endfunction

  function void write(my_item actual);
    my_item expected;
    bit match;
    if (exp_q.size() == 0) begin
      `uvm_error("SB", "无期望值") return;
    end
    expected = exp_q.pop_front();
```

```
    // 考点：比对前触发所有已注册的 pre_compare 回调
    `uvm_do_callbacks(extensible_scoreboard, sb_callback,
      pre_compare(expected, actual))

    match = (expected.data === actual.data);
    if (!match) `uvm_error("SB", "数据不匹配")

    // 考点：比对后触发所有已注册的 post_compare 回调
    `uvm_do_callbacks(extensible_scoreboard, sb_callback,
      post_compare(expected, actual, match))
  endfunction
endclass

// ===== 用户自定义回调（不修改 scoreboard）=====
class latency_check_cb extends sb_callback;
  `uvm_object_utils(latency_check_cb)
  function void post_compare(uvm_sequence_item exp,
                             uvm_sequence_item act, bit is_match);
    my_item e, a;
    $cast(e, exp); $cast(a, act);
    if ((a.timestamp - e.timestamp) > 100)
      `uvm_warning("CB", $sformatf(
        "事务延迟 %.1fns 超过 100ns 阈值", real'(a.timestamp - e.timestamp)))
  endfunction
endclass

// ===== Test 中注册回调（无需修改 scoreboard）=====
// latency_check_cb cb = latency_check_cb::type_id::create("cb");
// uvm_callbacks #(extensible_scoreboard, sb_callback)::add(sb_inst, cb);
```

【关键提示】

uvm_callback 实现了"开闭原则"：scoreboard 对修改封闭，对扩展开放。`uvm_register_cb 在 class 体内声明一次；`uvm_do_callbacks 在需要扩展点的地方调用；用户只需继承回调类并在 test 中注册实例，完全不需要触碰 scoreboard 源码。

题目 16 差分模型 Scoreboard（软件C模型对接）

难度：高级

标签：DPI-C / C参考模型 / PSNR / 图像验证 / 模糊比对

【题目描述】

验证一个图像处理加速器（例如双线性插值缩放或降噪滤波器）。由于算法逻辑复杂，参考模型用 C 语言实现并通过 DPI-C 接口供 SystemVerilog 调用。同时由于量化误差和算法近似，DUT 输出的像素值允许与参考值存在一定偏差，不能进行逐像素精确比对，而是采用峰值信噪比（PSNR）作为图像质量评价指标，PSNR 阈值可通过 uvm_config_db 进行配置。本题考察 DPI-C 函数的声明与调用约束、动态数组的传递方式，以及 PSNR 计算中的边界处理。

【考察重点与实现要点】

> DPI-C import 声明必须在 module/package 级别，不能在 class 或 function 内部
>
> open array 语法 byte unsigned in[]（不带维度）用于传递动态数组给 C 函数
>
> MSE 为 0.0 时 PSNR 为无穷大，必须特殊处理（返回 100.0），避免 log10(0) 崩溃
>
> write() 只入队，run_phase 中调用 C 函数，防止 write() 执行时间过长阻塞分析总线
>
> PSNR = 10 * log10(255^2 / MSE)，低于阈值（通常 30~40 dB）说明图像质量不足

【完整答案代码】

```
// 考点：DPI-C 声明必须在 module/package 级别，不能在 class 内
import "DPI-C" function void c_ref_model(
  input byte unsigned in_pixels[],
  input int width,
  input int height,
  output byte unsigned out_pixels[]
);

// 方法论：input（原始图像输入）→ analysis_fifo 缓存输入帧
// output（加速器输出）→ imp，write() 从 fifo 取输入帧，调用 C 模型计算期望，立即比对
class image_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(image_scoreboard)

  // 输入侧：analysis_fifo 缓存输入图像事务
  uvm_analysis_export #(img_item) input_export;
  uvm_tlm_analysis_fifo #(img_item) input_fifo;

  // 输出侧：imp，write() 中立即调用 C 参考模型并比对
  uvm_analysis_imp #(img_item, image_scoreboard) output_imp;

  real PSNR_THRESHOLD = 40.0;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    input_export = new("input_export", this);
    input_fifo = new("input_fifo", this);
    output_imp = new("output_imp", this);
    void'(uvm_config_db#(real)::get(this, "", "psnr_threshold", PSNR_THRESHOLD));
    `uvm_info("SB", $sformatf("PSNR 阈值: %.1f dB", PSNR_THRESHOLD), UVM_NONE)
  endfunction

  function void connect_phase(uvm_phase phase);
    input_export.connect(input_fifo.analysis_export);
  endfunction
```

```
  // output 端：write() 从 fifo 取输入帧，调用 C 参考模型计算期望输出，立即比对
  function void write(img_item act);
    img_item inp;
    byte unsigned ref_out[];
    real psnr_val;

    if (!input_fifo.try_get(inp)) begin
      `uvm_error("SB", "收到图像输出但输入 fifo 为空！") return;
    end

    // 考点：调用 DPI-C C 参考模型计算期望输出
    ref_out = new[inp.width * inp.height];
    c_ref_model(inp.input_pixels, inp.width, inp.height, ref_out);

    psnr_val = calc_psnr(act.output_pixels, ref_out);
    if (psnr_val < PSNR_THRESHOLD)
      `uvm_error("SB", $sformatf(
        "图像质量不足: PSNR=%.2f dB < 阈值 %.1f dB", psnr_val, PSNR_THRESHOLD))
    else
      `uvm_info("SB", $sformatf("PSNR=%.2f dB PASS", psnr_val), UVM_LOW)
  endfunction

  // 考点：MSE=0 时特殊处理，避免 log10(0) = -inf
  function real calc_psnr(byte unsigned actual[], byte unsigned expected[]);
    real mse = 0.0;
    foreach (actual[i])
      mse += (real'(actual[i]) - real'(expected[i])) ** 2;
    mse /= actual.size();
    if (mse == 0.0) return 100.0;
    return 10.0 * $log10(255.0 ** 2 / mse);
  endfunction

  function void check_phase(uvm_phase phase);
    if (input_fifo.size() != 0)
      `uvm_error("SB", $sformatf(
        "输入 fifo 有 %0d 帧未被输出侧消费！", input_fifo.size()))
  endfunction
endclass
```

【关键提示】

input（原始图像）→ analysis_fifo；output（加速器输出）→ imp。write() 中 try_get 取输入帧，立即调用 DPI-C C 参考模型计算期望输出并比对 PSNR。DPI-C import 声明必须在 module/package 级别，不能在 class 内。

题目 17 协议层叠 Scoreboard（数据包拆包验证）

难度：中级

标签：UDP/IP / 以太网 / 分层解析 / IP校验和 / 大端序

【题目描述】

验证一个 UDP/IP 数据包生成器。Monitor 在网络接口处捕获完整的原始字节流，Scoreboard 需要按照协议层次对字节流进行逐层解析与验证：第一层以太网帧头（14 字节），验证 EtherType 字段；第二层 IP 头（20 字节），验证版本号、协议号，并对 IP 头进行反码校验和验证；第三层 UDP 头（8 字节），验证源端口和目的端口；最后逐字节比对 payload 内容。本题考察分层协议解析的实现方式、网络大端序（big-endian）的字节拼接处理，以及 IP 头校验和的计算方法。

【考察重点与实现要点】

> 以太网和IP均为大端序（网络字节序）：eth_type = {bytes[12], bytes[13]}，高字节在前
> IP头校验和：取IP头20字节（10个16位字）反码求和，进位回卷后结果必须为 0xFFFF
> offset 指针逐层递增：以太帧头14B，IP头20B，UDP头8B，之后为payload
> 各层校验相互独立，任一层失败不跳过其他层检查
> exp_q 由上层（reference model 或 sequence）通过 add_expected() 注入期望值

【完整答案代码】

```
// 方法论：input（期望的 UDP 内容，由 sequence/ref_model 注入）→ analysis_fifo 缓存
// output（网络接口原始字节流）→ imp，write() 立即从 fifo 取期望值，解析并比对
class udp_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(udp_scoreboard)

  // 输入侧：analysis_fifo 缓存期望的 UDP 内容（由参考模型或 sequence 写入）
  uvm_analysis_export #(exp_udp_item) exp_export;
  uvm_tlm_analysis_fifo #(exp_udp_item) exp_fifo;

  // 输出侧：imp，write() 立即取期望值并解析比对
  uvm_analysis_imp #(raw_pkt_item, udp_scoreboard) raw_imp;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    exp_export = new("exp_export", this);
    exp_fifo = new("exp_fifo", this);
    raw_imp = new("raw_imp", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    exp_export.connect(exp_fifo.analysis_export);
  endfunction

  // output 端：write() 从 fifo 取期望值后立即解析比对，无需队列
  function void write(raw_pkt_item raw);
    exp_udp_item exp;
    if (!exp_fifo.try_get(exp)) begin
      `uvm_error("SB", "收到原始字节流但期望 fifo 为空！") return;
    end
    parse_and_check(raw, exp);
  endfunction

  function void parse_and_check(raw_pkt_item raw, exp_udp_item exp);
    int offset = 0;
    // 考点：以太网大端序，高字节 index 在前
```

```
    bit [15:0] eth_type = {raw.bytes[12], raw.bytes[13]};
    if (eth_type !== 16'h0800)
      `uvm_error("SB", $sformatf("EtherType 错误: exp=0x0800 act=0x%0h", eth_type))
    offset = 14;

    // IP 层
    bit [3:0] ip_ver = raw.bytes[offset][7:4];
    bit [7:0] ip_proto = raw.bytes[offset+9];
    if (ip_ver !== 4'h4) `uvm_error("SB", "IP 版本不是 IPv4")
    if (ip_proto !== 8'h11) `uvm_error("SB", "IP 协议字段不是 UDP (0x11)")
    check_ip_checksum(raw.bytes, offset);
    offset += 20;

    // UDP 层
    bit [15:0] src_port = {raw.bytes[offset], raw.bytes[offset+1]};
    if (src_port !== exp.src_port)
      `uvm_error("SB", $sformatf(
        "UDP 源端口不匹配: exp=%0d act=%0d", exp.src_port, src_port))
    offset += 8;

    // Payload 比对
    for (int i = 0; i < exp.payload.size(); i++) begin
      if (offset+i >= raw.bytes.size()) break;
      if (raw.bytes[offset+i] !== exp.payload[i])
        `uvm_error("SB", $sformatf(
          "UDP payload[%0d] 不匹配: exp=0x%0h act=0x%0h",
          i, exp.payload[i], raw.bytes[offset+i]))
    end
  endfunction

  // 考点：IP 头校验和，反码求和后结果必须为 0xFFFF
  function void check_ip_checksum(byte unsigned bytes[], int offset);
    int sum = 0;
    for (int i = 0; i < 10; i++)
      sum += {bytes[offset+i*2], bytes[offset+i*2+1]};
```

```
    while (sum >> 16) sum = (sum & 16'hFFFF) + (sum >> 16);
    if (sum !== 16'hFFFF)
      `uvm_error("SB", "IP 头校验和错误")
  endfunction

  function void check_phase(uvm_phase phase);
    if (exp_fifo.size() != 0)
      `uvm_error("SB", $sformatf(
        "期望 fifo 有 %0d 个条目未被消费！", exp_fifo.size()))
  endfunction
endclass
```

【关键提示】

期望的 UDP 内容（由参考模型或 sequence 写入）→ analysis_fifo；原始字节流（网络接口输出）→ imp。write() 中 try_get 取期望值后立即调用 parse_and_check()，无需任何队列或 run_phase。